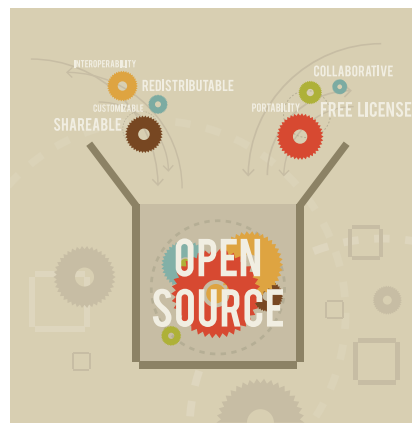
Why Open Source Dependencies Must be Managed

Unmanaged reliance on open source software may result in a support crisis over a project's life span, as well as financial loss for the organisation. Planned and regular upgrades of open source software components are a must.

Open source components are critical to large projects for a variety of reasons. They provide cost-effective solutions by removing licensing costs and encouraging community-based innovation. Furthermore, open source encourages cooperation, providing access to a wide reservoir of knowledge and various viewpoints. These components frequently have active communities that provide rapid updates, bug fixes, and security patches. Big projects can use open source components to expedite development, minimise time to market, and retain scalability.

When integrating open source components, however, use caution. To begin with, confirming licensing compliance is critical to avoiding legal problems. Second, reliance on other projects may present security vulnerabilities or compatibility difficulties, needing ongoing monitoring and upgrades. Third, because of the reliance on community assistance, replies to issues or bugs may be delayed and ineffective. Finally, due diligence is required in assessing the long-term maintenance of open source projects to reduce the risks associated with project abandonment or obsolescence.

In this article, we will look at the many issues of keeping open source components updated. Teams must be prepared to handle updates to diverse open source components without creating any downtime.



Everything starts from proof-of-concepts

In large-scale initiatives, proof-of-concept (PoC) is an important first stage in determining the feasibility and viability of suggested solutions. It facilitates decision-making by offering concrete proof of a concept's potential success. PoCs frequently include a prototype to test certain functionality or technologies. They enable technical feasibility testing prior to full-scale deployment.

While it is simple to leverage these open source components to create an initial proof-of-concept solution, disciplined execution and a thorough grasp of the open source components' release cycles are essential. Otherwise, having a working solution will not be useful.

Many developers make the error of not keeping track of the numerous open source components utilised, or even pinning the open source component version and continuing to use the solution, which is not a good practice and might have major effects if this system is put in production.

Functionality-based dependencies

A typical large-scale project may have certain dependencies on other open source software. These are summarised in Figure 1.

- **Operating system:** Projects are designed and developed with a specific OS/distribution in mind. This could be the operating system and a specific distribution of it, e.g., Ubuntu Linux.
- **Software platform:** This is a core dependency for any application. For example, some applications are delivered as containerised and may have platform dependency on Docker Compose or Kubernetes (K8), or any other container orchestration platform.
- **Library/modules:** Each programming language provides a way to import or reuse modules developed by other developers.
- **Complete subsystem:** Some projects may include some subsystems. Classic examples of this could be external subsystems like monitoring solutions (Prometheus, Grafana) or a database service.